

Installationsanleitung

RGB-Status-LED (WS2812B) + OLED-Display (SSD1306) für Raspberry Pi 4 ·
status_led.py

Diese Anleitung richtet eine Status-LED (WS2812B) und parallel ein 128×32-OLED (SSD1306 / Adafruit PiOLED) auf einem Raspberry Pi 4 ein. Die LED zeigt den Systemzustand per Farbe, das OLED parallel als Klartext (IP, CPU, RAM, Status). Beides steuert ein einziges Skript und ein Dienst. Am Ende findest du eine Praxis-Rubrik „Troubleshooting“ mit den Stolperfallen aus dem echten Aufbau.

Anzeige im Überblick

- **LED grün** — Normalbetrieb; hell aufflackernd bei Disk-Aktivität
- **LED rot/grün im Sekundentakt** — Übertemperatur (Schwelle konfigurierbar)
- **LED blau blinkend** — keine Netzwerkverbindung
- **LED magenta blinkend** — Backup fehlgeschlagen
- **LED cyan pulsierend** — Backup läuft
- **OLED** — zeigt durchgehend IP-Adresse, CPU-Temperatur + Last, RAM-Auslastung und den Zustand im Klartext

1. Voraussetzungen

Hardware

- Raspberry Pi 4 mit Raspberry Pi OS (Bookworm) — **mit physischem GPIO-Header**
- WS2812B-LED (5050), einzeln oder als Streifen
- OLED-Display 128×32, SSD1306, I2C (z. B. Adafruit PiOLED)
- Widerstand 330–470 Ω für die LED-Datenleitung
- Verbindungskabel; für mehrere LEDs zusätzlich ein Pegelwandler (74AHCT125) und ein 1000 µF Elko

Software

- Python 3 (bei Raspberry Pi OS vorinstalliert)
- Internetzugang für die einmalige Installation der Bibliotheken
- Die Datei status_led.py (aus dem ZIP)

2. LED-Anschlussmethode: PWM oder SPI

Die WS2812B lässt sich auf zwei Wegen ansteuern. Das OLED ist davon unberührt (eigener Bus).

- **PWM (GPIO18) — empfohlen**, besonders für eine einzelne LED. Die Datenleitung wird sauber getrieben und in Ruhe auf Low gezogen — „Aus“ bleibt aus und schwache Farben (dim grün) bleiben stabil. Voraussetzung: Dienst läuft als **root** und das Onboard-Audio muss deaktiviert werden.
- **SPI (GPIO10/MOSI)** — läuft ohne root und ohne Audio-Konflikt. **Aber:** der Pi lässt die SPI-Datenleitung zwischen den Paketen auf High; manche WS2812B lesen das als „alle Bits an“ = Weiß. Dim-Farben und „Aus“ können dann nach Weiß driften (siehe Troubleshooting). Nur nutzen, wenn root/Audio ein Problem sind und die LED den Test besteht.

Tipp: Im Zweifel PWM nehmen — das ist der robuste Standardweg für eine einzelne WS2812B am Pi. Diese Anleitung ist auf PWM ausgerichtet; die SPI-Schritte stehen jeweils als Alternative dabei.

3. Verdrahtung (LED + OLED)

Pi ausschalten und beides anschließen. Die Pins der zwei Geräte überschneiden sich nicht:

WS2812B (LED)

- **DIN** (Pfeilrichtung beachten — Eingangsseite) → über den **330-470 Ω Widerstand** an **GPIO18 / Pin 12** (PWM) bzw. GPIO10/MOSI (SPI)
- **VDD** → bei einer **einzelnen** LED an **3,3 V (Pin 1)**; bei mehreren LEDs an 5 V mit Pegelwandler
- **GND** → GND des Pi (gemeinsame Masse ist Pflicht)

Tipp: VDD an 3,3 V statt 5 V ist bei einer einzelnen LED die einfachste und saubere Dauerlösung: Der Pi liefert die Daten mit 3,3 V, eine WS2812B an 5 V verlangt aber ~3,5 V für „High“ — an 3,3 V-VDD passt der Pegel wieder. Für mehrere LEDs (5 V nötig) gehört ein Pegelwandler (74AHCT125) dazu.

OLED (SSD1306, I2C)

- **3V3** → Pin 1 · **GND** → GND
- **SDA** → GPIO2 (Pin 3) · **SCL** → GPIO3 (Pin 5)

Taster (Display + Neustart)

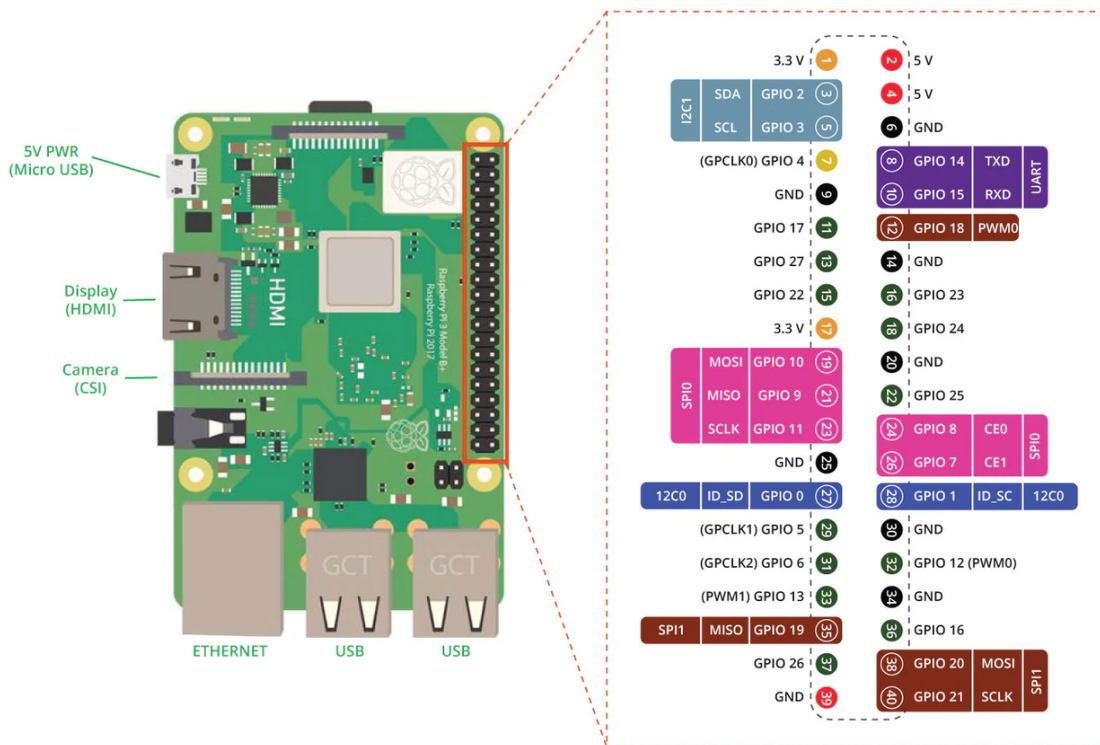
- Ein Bein → **GPIO17 (Pin 11)**, das andere Bein → **GND** (z. B. Pin 14)
- Interner Pull-up per Software (gedrückt = LOW), daher **kein Widerstand nötig**
- Kurzer Druck schaltet die Display-Seiten weiter, langer Druck (≥ 5 s) startet den Pi neu

Kombinierte Pin-Übersicht

Gerät / Funktion	Pin	Signal
LED – Daten (PWM, empf.)	Pin 12	GPIO18
LED – Daten (SPI, alt.)	Pin 19	GPIO10 / MOSI
LED – VDD (1 LED)	Pin 1	3V3
LED – GND	Pin 9 (frei)	GND
OLED – 3V3	Pin 1	3V3
OLED – SDA	Pin 3	GPIO2
OLED – SCL	Pin 5	GPIO3
OLED – GND	Pin 6	GND
Taster – Signal	Pin 11	GPIO17
Taster – GND	Pin 14 (frei)	GND

Pin 1 (3V3) versorgt sowohl OLED als auch die einzelne LED — beide dürfen daran hängen. Eine PiOLED steckt meist auf den Eckpins (1/3/5 + GND); nimm GND der LED daher von einem freien Pin (z. B. Pin 9).

Raspberry Pi 4 - Pinout (Referenz)



Belegung für dieses Projekt: LED-Daten an GPIO18 (Pin 12), LED-VDD und OLED-3V3 an 3,3 V (Pin 1), OLED-SDA an GPIO2 (Pin 3), OLED-SCL an GPIO3 (Pin 5), Taster an GPIO17 (Pin 11) gegen GND, gemeinsame Masse an einem GND-Pin (z. B. Pin 6, 9 oder 14). SPI-Alternative für die LED-Daten: GPIO10/MOSI (Pin 19).

4. Funktionieren OLED und LED zusammen?

Ja. Das OLED hängt am **I2C-Bus** (GPIO2/3), die LED an der **PWM-Leitung** (GPIO18) bzw. am **SPI-Bus** (GPIO10). Getrennte Hardware-Busse auf verschiedenen Pins — kein Konflikt. Beide werden vom selben Prozess gesteuert (status_led.py aktualisiert das OLED 1x/Sekunde). Schlägt die OLED-Initialisierung fehl, läuft die LED trotzdem weiter, und umgekehrt.

Voraussetzung: echte GPIO-/I2C-/SPI-Schnittstellen eines Raspberry Pi. In einem LXC-Container oder einer VM (z. B. auf einem x86-Proxmox-Host) fehlt der Pi-Header — dann funktioniert nichts davon. Schnellprüfung, ob es ein echter Pi mit beiden Bussen ist:

```
cat /proc/device-tree/model      # muss 'Raspberry Pi 4 ...' zeigen
ls -l /dev/i2c-1 /dev/spidev0.0 # beide Geraetedateien muessen da sein
```

5. Skript ablegen

Die Datei aus dem ZIP in ein festes Verzeichnis kopieren und ausführbar machen:

```
sudo cp status_led.py /usr/local/bin/status_led.py
sudo chmod +x /usr/local/bin/status_led.py
```

6. Bibliotheken installieren

Jeder Befehl steht bewusst in **einer Zeile** — ein Backslash am Zeilenende ist nicht nötig (und führt, mitten in der Zeile, zu Fehlern).

PWM (empfohlen)

```
sudo apt install -y python3-pil i2c-tools
sudo pip3 install adafruit-blinka --break-system-packages
sudo pip3 install rpi_ws281x --break-system-packages
sudo pip3 install adafruit-circuitpython-neopixel --break-system-packages
sudo pip3 install adafruit-circuitpython-ssd1306 --break-system-packages
```

SPI (Alternative)

```
sudo apt install -y python3-pil i2c-tools
sudo pip3 install adafruit-blinka --break-system-packages
sudo pip3 install adafruit-circuitpython-neopixel-spi --break-system-packages
sudo pip3 install adafruit-circuitpython-ssd1306 --break-system-packages
```

Alternative — virtuelle Umgebung (ohne System-Python anzufassen)

```
sudo apt install -y python3-venv i2c-tools
sudo python3 -m venv /opt/status-led-venv
sudo /opt/status-led-venv/bin/pip install adafruit-blinka rpi_ws281x
sudo /opt/status-led-venv/bin/pip install adafruit-circuitpython-neopixel
sudo /opt/status-led-venv/bin/pip install adafruit-circuitpython-ssd1306 pillow
```

Bei der venv-Variante im Dienst (Abschnitt 10) den Interpreter aus dem venv verwenden:
ExecStart=/opt/status-led-venv/bin/python /usr/local/bin/status_led.py

7. I2C aktivieren, OLED prüfen, (PWM:) Audio deaktivieren

I2C für das OLED aktivieren (für die SPI-LED zusätzlich SPI):

```
sudo raspi-config
# Interface Options -> I2C -> Yes
# (nur SPI-Variante: Interface Options -> SPI -> Yes)
```

Bei der **PWM-Variante** muss das Onboard-Audio aus (es teilt sich die PWM-Hardware, sonst flackert die LED). In /boot/firmware/config.txt:

```
sudo sed -i 's/^dtparam=audio=on/dtparam=audio=off/' /boot/firmware/config.txt
grep audio /boot/firmware/config.txt
# zeigt grep kein 'audio=off', dann einmalig:
echo "dtparam=audio=off" | sudo tee -a /boot/firmware/config.txt
```

Nach den Interface-Änderungen neu starten und das OLED auf dem I2C-Bus suchen:

```
sudo reboot
# nach dem Neustart:
sudo i2cdetect -y 1
```

Erscheint in der Tabelle '3c', ist das Display erkannt. Zeigt deins 0x3d, diesen Wert in der Config bei oled_addr eintragen.

8. Konfiguration anpassen

Den Konfigurationsblock am Anfang von `status_led.py` öffnen (Auszug):

```
sudo nano /usr/local/bin/status_led.py

@dataclass
class Config:
    led_type: str = "ws2812"          # PWM (empfohlen); SPI: "ws2812-spi"
    ws_count: int = 1                 # tatsächliche Anzahl LEDs!
    ws_brightness: float = 0.50       # Master-Helligkeit 0..1
    ws_pixel_order: str = "GRB"       # Standard-WS2812B; selten "RGB"
    oled_enabled: bool = True         # OLED parallel ansteuern
    oled_addr: int = 0x3C              # I2C-Adresse (siehe i2cdetect)
    temp_threshold_c: float = 70.0    # Schwelle Ubertemperatur in Grad C
    net_check_host: str = "1.1.1.1"   # Internet-Check; fuer LAN die Gateway-IP
    button_enabled: bool = True       # Taster an GPIO17
    button_pin: int = 17              # BCM-Pin, Taster gegen GND (Pull-up)
    button_long_press_s: float = 5.0  # so lange halten -> Neustart
    oled_page_timeout_s: float = 30.0 # Auto-Ruecksprung auf Seite 0
```

- **led_type** — "ws2812" (PWM) oder "ws2812-spi" (SPI)
- **ws_count** — **genau** die Anzahl deiner LEDs; zu niedrig → überzählige LEDs bleiben auf altem Wert
- **ws_pixel_order** — fast immer "GRB"; nur ändern, wenn der Farbttest es zeigt
- **ws_brightness** — Master-Dimmer; WS2812B sind sehr hell, 0.2-0.5 ist meist angenehm
- **button_pin** — BCM-Pin des Tasters (Standard GPIO17 / Pin 11)
- **button_long_press_s** — Haltedauer, die einen Neustart auslöst (Standard 5 s)
- **oled_page_timeout_s** — nach dieser Untätigkeit springt das Display zurück zur Übersicht

9. Funktionstest

Ohne Hardware (Terminal)

Zeigt LED-Zustände und OLED-Zeilen parallel im Terminal:

```
python3 status_led.py --simulate --duration 30
```

LED-Direkttest (Farben + Stabilität, PWM)

Prüft Farbreihenfolge und ob dim grün stabil bleibt (mit `sudo`, GPIO18):

```
sudo systemctl stop status-led.service
sudo python3 - <<'PYEOF'
import board, neopixel, time
px = neopixel.NeoPixel(board.D18, 1, brightness=0.3, auto_write=True, pixel_order="GRB")
for n,c in [("ROT",(255,0,0)),("GRUEN",(0,255,0)),("BLAU",(0,0,255))]:
    px.fill(c); print("Befohlen:", n); time.sleep(2)
print("20s dim gruen - bleibt gruen, dann sauber aus?")
for i in range(200): px.fill((0,40,0)); time.sleep(0.1)
px.fill((0,0,0))
PYEOF
```

Rot/Grün/Blau müssen passen, das dim Grün muss stabil bleiben und am Ende sauber ausgehen. Für die SPI-Variante stattdessen `neopixel_spi.NeoPixel_SPI(board.SPI(), 1, ...)` verwenden.

Dienst im Vordergrund (zeigt Fehler live)

```
python3 /usr/local/bin/status_led.py    # Beenden mit Strg+C
```

10. Autostart als Dienst (systemd)

Ein Dienst steuert LED und OLED gemeinsam. Dienst-Datei anlegen (Vorlage liegt im ZIP):

```
sudo nano /etc/systemd/system/status-led.service
```

```
[Unit]
Description=RGB Status LED + OLED
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/bin/python3 /usr/local/bin/status_led.py
Restart=always
RestartSec=5
User=root
RuntimeDirectory=status-led
# SPI-Variante: User=DEIN_BENUTZERNAME (NICHT 'pi')
# venv: ExecStart=/opt/status-led-venv/bin/python /usr/local/bin/status_led.py

[Install]
WantedBy=multi-user.target
```

Tipp: Wichtig beim Benutzer: **PWM braucht User=root**. Bei der SPI-Variante einen echten Benutzernamen eintragen — auf aktuellem Raspberry Pi OS gibt es den früheren Standard-User pi nicht mehr. Ein falscher/fehlender User führt zu status=217/USER (siehe Troubleshooting).

Aktivieren, starten und kontrollieren:

```
sudo systemctl daemon-reload
sudo systemctl enable --now status-led.service
systemctl status status-led.service
journalctl -u status-led -f
```

Bei der SPI-Variante als normaler Benutzer muss dieser in den Gruppen 'spi' und 'i2c' sein (Standard auf Raspberry Pi OS): `groups DEIN_BENUTZER bzw. sudo usermod -aG spi,i2c DEIN_BENUTZER`.

11. Display weiterschalten und Neustart per Taster

Ein Taster an **GPIO17 (Pin 11)** steuert das Display und kann den Pi neu starten:

- **Kurzer Druck** — schaltet eine Seite weiter. Seite 0 ist die gewohnte 4-Zeilen-Übersicht; die Seiten 1–4 zeigen je eine Zeile (IP, CPU, RAM, Status) einzeln in großer, automatisch eingepasster Schrift. Nach der letzten Seite geht es zurück zur Übersicht.
- **Auto-Rücksprung** — nach etwa 30 s ohne Tastendruck springt das Display zurück auf Seite 0 (einstellbar über `oled_page_timeout_s`).
- **Langer Druck (≥ 5 s)** — startet den Pi neu: OLED zeigt „Neustart...“, LED wird rot, dann läuft `systemctl reboot` (Haltedauer über `button_long_press_s`).

Der Neustart braucht Root-Rechte — beim empfohlenen PWM-Setup läuft der Dienst ohnehin als `User=root`, funktioniert also direkt. Bei der SPI-Variante unter einem normalen Benutzer wäre eine `sudoers`-/polkit-Regel nötig. Der Taster nutzt Blinks digitalio und braucht kein

zusätzliches Paket; abschalten mit --no-button.

12. Backup-Status anbinden (optional)

Das Skript liest den Backup-Zustand aus /run/status-led/backup. Fehlt die Datei, wird kein Backup-Zustand angezeigt. Der Backup-Job schreibt running, ok oder failed hinein:

```
#!/bin/bash
mkdir -p /run/status-led
echo running > /run/status-led/backup

if restic backup /daten; then
    echo ok > /run/status-led/backup
else
    echo failed > /run/status-led/backup
fi
```

Der Fehlerzustand (LED magenta, OLED „Backup-Fehler!“) bleibt, bis die Datei wieder auf 'ok' steht.

13. Referenz: Zustände, LED-Farbe und OLED-Text

Zustand	LED-Farbe & Muster	OLED-Text	Prio
Übertemperatur	Rot/Grün, 1 Hz	UEBERTEMPERATUR!	100
Kein Netzwerk	Blau, 2 Hz blinkend	Kein Netzwerk	80
Backup fehlgeschlagen	Magenta, 2 Hz	Backup-Fehler!	70
Backup läuft	Cyan, pulsierend	Backup laeuft...	40
Normalbetrieb	Grün (hell bei I/O)	Normalbetrieb	0

Bei mehreren aktiven Zuständen gewinnt der mit der höchsten Priorität (für LED-Farbe und OLED-Text gleichermaßen). Das OLED zeigt zusätzlich dauerhaft IP, CPU-Temperatur + 1-Minuten-Last und RAM.

14. Eigene Zustände ergänzen

Jeder Zustand besteht aus Bedingung und Render-Funktion und wird mit Priorität in der Liste STATUSES registriert. Für die OLED-Anzeige zusätzlich einen Text in STATUS_TEXT:

```
def is_mein_zustand(ctx):
    return ...                # True, wenn aktiv

def render_mein_zustand(ctx):
    return (1.0, 0.5, 0.0)    # Farbe (R, G, B), je 0..1

STATUSES.append(StatusDef("mein_zustand", priority=50,
                           condition=is_mein_zustand,
                           render=render_mein_zustand))
STATUS_TEXT["mein_zustand"] = "Mein Text"
```


15. Troubleshooting (aus der Praxis)

Die folgenden Fälle stammen aus einer echten Inbetriebnahme — vom Dienststart bis zur flackernden LED:

► Dienst startet nicht, Status „status=217/USER“ im Log

Ursache: Der in der .service-Datei angegebene Benutzer existiert nicht. Auf aktuellem Raspberry Pi OS gibt es den früheren Standard-User pi nicht mehr.

Lösung: Bei PWM User=root setzen; bei SPI den echten Benutzernamen. Dann daemon-reload und restart.

► pip-Fehler „externally-managed-environment“

Ursache: Auf Bookworm ist das System-Python geschützt (PEP 668) — oder ein Backslash steht mitten in der Zeile statt am Zeilenende und zerlegt den Befehl.

Lösung: Jeden Befehl in **eine** Zeile schreiben und --break-system-packages ans Ende setzen, oder die venv-Variante (Abschnitt 6) verwenden.

► LED leuchtet gar nicht, obwohl das Skript ohne Fehler läuft

Ursache: Verkabelung (DIN/DOUT vertauscht, keine gemeinsame Masse) oder zu niedriger Logikpegel (VDD an 5 V, Daten nur 3,3 V).

Lösung: Zuerst mit dem Direkttest (Abschnitt 9) volle Helligkeit prüfen. Leuchtet nichts: DIN-Seite und GND prüfen, dann VDD probeweise von 5 V auf **3,3 V** umstecken. Bei einer einzelnen LED ist 3,3 V die Dauerlösung.

► Falsche Farben (z. B. Rot befohlen, Grün leuchtet)

Ursache: Die Farbkanal-Reihenfolge der LED weicht ab. Standard ist GRB; manche Chargen sind anders.

Lösung: Mit dem Farbttest (Abschnitt 9) ermitteln, bei welcher Reihenfolge „Rot“ wirklich rot ist, und diesen Wert in ws_pixel_order eintragen. Meist ist GRB korrekt.

► LED driftet in Ruhe nach Weiß, „Aus“ bleibt nicht aus (bei SPI)

Ursache: Der Pi lässt die SPI-Datenleitung (MOSI) zwischen den Paketen auf High. Die WS2812B liest das als lauter Einsen = Weiß. Helle Farben überstehen es, dim Grün und „Aus“ kippen.

Lösung: Auf die **PWM-Methode (GPIO18)** wechseln — sie zieht die Leitung sauber auf Low. Datenleitung auf Pin 12 umstecken, led_type="ws2812", Audio aus, Dienst als root (Abschnitte 6–10).

► Nur eine von mehreren LEDs zeigt den Status, der Rest bleibt (weiß)

Ursache: ws_count ist zu niedrig — überzählige LEDs behalten ihren letzten Wert.

Lösung: ws_count auf die tatsächliche Anzahl LEDs setzen und Dienst neu starten.

► PWM: LED flackert oder zeigt falsche Farben trotz richtiger Verkabelung

Ursache: Das Onboard-Audio ist noch aktiv und belegt die PWM-Hardware.

Lösung: dtparam=audio=off in /boot/firmware/config.txt eintragen und neu starten (Abschnitt 7).

► OLED bleibt dunkel

Ursache: I2C nicht aktiviert, falsche Adresse, PIL fehlt oder Verkabelung.

Lösung: sudo i2cdetect -y 1 — erscheint 0x3c? Sonst I2C aktivieren, python3-pil/pillow installieren, SDA/SCL/3V3/GND prüfen; ggf. 0x3d in oled_addr.

► Weder LED noch OLED reagieren; /dev/i2c-1 oder /dev/spidev0.0 fehlen

Ursache: Kein echter Raspberry Pi (z. B. LXC-Container/VM auf x86) oder Bus nicht aktiviert.

Lösung: cat /proc/device-tree/model muss einen Pi zeigen. In LXC/VM ohne durchgereichte GPIO-/I2C-/SPI-Schnittstelle gibt es keine Pins — dann auf echter Pi-Hardware betreiben.

► **Nach einem System-Update flackert die LED plötzlich wieder (PWM)**

Ursache: Ein Update hat /boot/firmware/config.txt zurückgesetzt; dtparam=audio=off fehlt wieder.

Lösung: Die Zeile erneut eintragen und neu starten.

► **Taster reagiert nicht**

Ursache: Falscher Pin, Taster nicht gegen GND verdrahtet oder deaktiviert.

Lösung: Taster zwischen GPIO17 (Pin 11) und GND anschließen, button_pin und button_enabled = True prüfen. Interner Pull-up: gedrückt = LOW, kein Widerstand nötig.

► **Langer Druck löst keinen Neustart aus**

Ursache: Der Dienst läuft nicht als root (z. B. SPI-Variante unter normalem Benutzer).

Lösung: Dienst als User=root betreiben (PWM-Standard), oder eine sudoers-/polkit-Regel für systemctl reboot anlegen.

► **Die großen Einzelseiten zeigen winzige Schrift**

Ursache: Kein skalierbarer TrueType-Font gefunden, daher wird der kleine Bitmap-Font verwendet.

Lösung: DejaVu-Fonts installieren: `sudo apt install -y fonts-dejavu-core`.

Nützliche Diagnose-Befehle

```
cat /proc/device-tree/model          # echter Pi?
ls -l /dev/i2c-1 /dev/spidev0.0     # Busse vorhanden?
sudo i2cdetect -y 1                  # OLED unter 0x3c?
groups DEIN_BENUTZER                 # in Gruppen spi/i2c?
python3 /usr/local/bin/status_led.py # (Dienst stoppen) Fehler im Vordergrund
journalctl -u status-led -e          # Dienst-Log mit Fehlern
```